

Multiprocessing

Основные инструменты

Модуль Multiprocessing

- ▶ Создание процессов и управление ими
- ▶ Структуры данных для информационного обмена между процессами
- ▶ Классы для синхронизации процессов
- ▶ Документация <https://docs.python.org/3/library/multiprocessing.html>

Пример создания процесса

```
from multiprocessing import Process

def f(name):
    print('hello', name)

if __name__ == '__main__':
    p = Process(target=f, args=('bob',))
    p.start()
    p.join()
```

▶ Name == '__main__' - обозначение главного процесса

▶ Start, join - аналогично работе с потоками

▶ Параметры процесса

Process(group=None, target=None, name=None, args=(), kwargs={}, *, daemon=None)

▶ Документация

<https://docs.python.org/3/library/multiprocessing.html#multiprocessing.Process>

Пример с передачей аргументов

```
from multiprocessing import Process
import os

def info(title):
    print(title)
    print('module name:', __name__)
    print('parent process:', os.getppid())
    print('process id:', os.getpid())

def f(name):
    info('function f')
    print('hello', name)

if __name__ == '__main__':
    info('main line')
    p = Process(target=f, args=('bob',))
    p.start()
    p.join()
```

Processing Pool

- ▶ Создание нескольких процессов, выполняющих одну и ту же функцию
- ▶ Автоматическое распределение данных между процессами
- ▶ Централизованный результат
- ▶ <https://docs.python.org/3/library/multiprocessing.html#multiprocessing.pool.Pool>

```
from multiprocessing import Pool
import time

def f(x):
    return x*x

if __name__ == '__main__':
    with Pool(processes=4) as pool:        # start 4 worker processes
        result = pool.apply_async(f, (10,)) # evaluate "f(10)" asynchronously in a single process
        print(result.get(timeout=1))        # prints "100" unless your computer is *very* slow

        print(pool.map(f, range(10)))      # prints "[0, 1, 4, ..., 81]"

        it = pool.imap(f, range(10))
        print(next(it))                    # prints "0"
        print(next(it))                    # prints "1"
        print(it.next(timeout=1))          # prints "4" unless your computer is *very* slow

        result = pool.apply_async(time.sleep, (10,))
        print(result.get(timeout=1))        # raises multiprocessing.TimeoutError
```

Manager для управления данными

- ▶ Разделяемая память
- ▶ Защита доступа
- ▶ list, dict, Namespace, Lock, RLock, Semaphore, BoundedSemaphore, Condition, Event, Barrier, Queue, Value, Array.

<https://docs.python.org/3/library/multiprocessing.html#managers>

```
from multiprocessing import Process, Manager

def f(d, l):
    d[1] = '1'
    d['2'] = 2
    d[0.25] = None
    l.reverse()

if __name__ == '__main__':
    with Manager() as manager:
        d = manager.dict()
        l = manager.list(range(10))

        p = Process(target=f, args=(d, l))
        p.start()
        p.join()

        print(d)
        print(l)
```

Queue - безопасная очередь

```
from multiprocessing import Process, Queue

def f(q):
    q.put([42, None, 'hello'])

if __name__ == '__main__':
    q = Queue()
    p = Process(target=f, args=(q,))
    p.start()
    print(q.get())      # prints "[42, None, 'hello']"
    p.join()
```

<https://docs.python.org/3/library/queue.html#queue.Queue>

Pipe - обмен сообщениями

```
from multiprocessing import Process, Pipe

def f(conn):
    conn.send([42, None, 'hello'])
    conn.close()

if __name__ == '__main__':
    parent_conn, child_conn = Pipe()
    p = Process(target=f, args=(child_conn,))
    p.start()
    print(parent_conn.recv())    # prints "[42, None, 'hello']"
    p.join()
```

<https://docs.python.org/3/library/multiprocessing.html#multiprocessing.Pipe>